Rechnernetze und Netzwerkprogrammierung
Übungsblatt 01

Raphael Bleier

Die sieben Programme liegen in diesem Ordner. Sie brauchen nur die Python-Standardbibliothek. Das Aufgabenblatt nennt Python 3; die Einführungsfolien empfehlen Version 3.8.5 oder neuer.

Getestet wurde mit Python 3.8.20.

```
uv run --python 3.8.20 python -m unittest discover -s tests -v
python threaded_calc.py zahlen.txt --threads 3
python thread_safety.py zahlen.txt --threads 4 --repeats 3
python producer_consumer.py --count 20 --queue-size 3
```

1.1 Python-Umgebung

`hello.py` gibt Name, Python-Version und das aktuelle Arbeitsverzeichnis aus. Die virtuelle Umgebung hält die Python-Umgebung eines Projekts getrennt von anderen Installationen:

```
python3 -m venv .venv
source .venv/bin/activate
python hello.py
```

1.2 Text, Zeichenketten und Bytes

`str` ist eine Zeichenkette mit Unicode-Zeichen. `bytes` ist eine Folge von Bytewerten. UTF-8 braucht je nach Zeichen unterschiedlich viele Bytes. Zum Beispiel hat `ä` in einem Python-`str` die Länge 1, seine UTF-8-Darstellung besteht aber aus zwei Bytes (`c3 a4`). Darum können `len(text)` und `len(text.encode("utf-8"))` verschieden sein.

Wird eine Bytefolge mit einer falschen Kodierung gelesen, kann das einen Dekodierfehler auslösen oder zu falschen Zeichen führen. Bei Netzwerkprogrammen ist die Unterscheidung wichtig, weil übertragen werden Bytes. Text wird vor dem Senden kodiert und nach dem Empfang mit der passenden Kodierung dekodiert.

1.3 Datei-I/O

`convert.py` bekommt Ein- und Ausgabedatei als Kommandozeilenparameter. Beim Lesen mit `newline=None` wandelt Python LF, CRLF und CR in ein einheitliches `\n` um. Das Programm schreibt anschließend LF, unabhängig vom Betriebssystem. `with` schließt die Dateien auch dann, wenn beim Lesen oder Schreiben ein Fehler auftritt.

Fehlende Parameter meldet `argparse`. Nicht vorhandene, nicht lesbare oder ungültig kodierte Eingaben sowie nicht beschreibbare Ausgaben führen zu einer Fehlermeldung. Eingabe- und Ausgabepfad dürfen nicht identisch sein, damit die Quelldatei nicht überschrieben wird.

1.4 Binäre Daten und Protokollnachrichten

Die ersten 32 Bit sind so aufgeteilt: Version in Bit 0–4 (Wert 2), reservierte Bits in Bit 5–13 (alle 0), D-Flag in Bit 14, U-Flag in Bit 15 und Sequenznummer in Bit 16–31. Danach folgen die 32-Bit-Payload-Länge und die Payload. `struct`-Format `!II` kodiert den 32-Bit-Wert mit Flags und Sequenznummer sowie die 32-Bit-Payload-Länge in Network Byte Order (Big Endian).

`create_message` prüft Flags, Sequenznummer und Payload. `parse_message` prüft Version, reservierte Bits, positive Sequenznummer und ob die angegebene Payload-Länge genau zur Nachricht passt. Der Unit-Test vergleicht zusätzlich einen konkreten Byte-String und prüft fehlerhafte Eingaben.

1.5 Nebenläufigkeit mit Threads

Die Werte werden in zusammenhängende, überschneidungsfreie Bereiche aufgeteilt. Mit `divmod` bekommt jeder Thread zunächst gleich viele Elemente; die ersten Threads erhalten zusätzlich je ein Element, bis der Rest verteilt ist. So wird jeder Arraywert genau einmal verarbeitet, auch wenn die Länge nicht durch die Threadanzahl teilbar ist.

`Thread(...)` erzeugt ein Thread-Objekt. Erst `start()` startet dessen Funktion; `join()` wartet, bis sie fertig ist. Jeder Worker schreibt sein Teilergebnis in einen eigenen Eintrag der Ergebnisliste. So müssen die Threads die Summe nicht gleichzeitig ändern. Schlägt ein Threadstart fehl, wartet das Programm nur auf bereits gestartete Threads; Workerfehler werden nach dem Einsammeln weitergereicht.

Die Berechnung verwendet viele Quadratwurzeln als Beispiel für wiederholte Arbeit. Bei CPU-lastigem Python-Code beschleunigen Threads CPython wegen des Global Interpreter Lock nicht automatisch. Hier geht es um das Aufteilen der Arbeit und das Warten auf alle Worker.

1.6 Thread Safety

Thread Safety bedeutet, dass gleichzeitige Zugriffe keine falschen oder inkonsistenten Ergebnisse erzeugen. `result += value` ist als Lesen, Addieren und Schreiben zu verstehen. Wenn zwei Threads denselben alten Wert lesen, kann einer den Beitrag des anderen überschreiben.

Das Programm zeigt zuerst diesen verlorenen Update-Fall. Eine Barriere sorgt dafür, dass alle Threads vor dem Schreiben denselben alten Wert gelesen haben. Im zweiten Durchlauf schützt `Lock` das Ändern der gemeinsamen Summe. Der Lock bleibt nur während dieses kurzen kritischen Abschnitts aktiv. Das Programm führt den Vergleich standardmäßig dreimal aus und gibt dazu die sequenzielle Sollsumme aus. Bei Workerfehlern wird die Barriere abgebrochen; Startfehler sammeln bereits gestartete Threads ein.

1.7 Producer-Consumer

`queue.Queue` ist für mehrere Threads geeignet, weil Einfügen und Entnehmen synchronisiert werden. Bei einer begrenzten Queue wartet `put()`, wenn sie voll ist; `get()` wartet, wenn sie leer ist. Ein Sentinel markiert das Ende der Produktion. `task_done()` meldet erledigte Einträge und `join()` wartet, bis die Queue abgearbeitet ist. Kurze Timeouts lassen die Worker auf Fehler reagieren; diese werden im aufrufenden Thread gemeldet. Bei Fehlern wird nicht auf unerledigte Einträge gewartet, sondern es werden die gestarteten Threads eingesammelt. Eine normale Liste bietet diese Synchronisation nicht.

KI-Nutzung: Hermes Agent, OpenAI GPT-6 Luna (gpt-6-luna), 12.10.2026. Die verwendeten Aufgabenblatt-Auszüge stehen als Anfrage in den Kommentaren am Anfang der Python-Dateien.